

Day 15

System Verilog: Classes and OOP concepts

FIFO using class:

```
class fifo_txn;
  logic [7:0] data;
  logic wr_en;
  logic rd_en;

  function new();
    data = 0;
    wr_en = 0;
    rd_en = 0;
  endfunction

  function void print();
    $display("Transaction: data=%h, wr=%b, rd=%b", data, wr_en, rd_en);
  endfunction
endclass
```

Testbench:

```
module tb;
  initial begin
    fifo_txn txn;
    txn = new();

    txn.data = 8'hAA;
    txn.wr_en = 1;

    txn.print();

  end
endmodule
```

Output:

Transaction: data=aa, wr=1, rd=0

Using rand keyword to randomize the stimulus

```
class fifo_txn;
  rand logic [7:0] data;
  rand logic wr_en;
  rand logic rd_en;

  constraint rw_exclusive {
    !(wr_en && rd_en); }

  constraint nonzero {
    data != 0; }
  function new();
    data = 0;
    wr_en = 0;
    rd_en = 0;
  endfunction
endclass
```

```
endfunction
```

```
function void print();  
$display("Transaction: data=%h, wr=%b, rd=%b", data, wr_en, rd_en);  
endfunction  
endclass
```

Testbench:

```
module tb;  
initial begin  
    fifo_txn txn;  
    txn = new();  
  
    repeat(10) begin  
        txn.randomize();  
        txn.print();  
    end  
end  
endmodule
```

Output:

```
# KERNEL: Transaction: data=be, wr=1, rd=0  
# KERNEL: Transaction: data=0e, wr=0, rd=1  
# KERNEL: Transaction: data=f4, wr=0, rd=1  
# KERNEL: Transaction: data=b0, wr=1, rd=0  
# KERNEL: Transaction: data=2e, wr=1, rd=0  
# KERNEL: Transaction: data=89, wr=1, rd=0  
# KERNEL: Transaction: data=18, wr=0, rd=1  
# KERNEL: Transaction: data=e1, wr=0, rd=0  
# KERNEL: Transaction: data=85, wr=0, rd=1  
# KERNEL: Transaction: data=70, wr=0, rd=1
```

Polymorphism:

```
// Code your design here  
class fifo_txn;  
    rand logic [7:0] data;  
    rand logic    wr_en;  
  
    function new();  
        data = 0;  
        wr_en = 0;  
    endfunction  
  
    // virtual means child classes can override this  
    virtual function void print();  
        $display("Base txn: data=%h wr=%b", data, wr_en);  
    endfunction  
endclass  
  
class fifo_burst_txn extends fifo_txn;  
    rand int unsigned burst_len;  
  
    constraint burst_range {  
        burst_len inside {[1:8]};  
    }
```

```

function void print();
    $display("Burst txn: data=%h wr=%h len=%0d", data, wr_en, burst_len);
endfunction
endclass

```

Testbench:

```

module tb;
    initial begin
        fifo_txn base;
        fifo_burst_txn burst;
        fifo_txn poly_handle;

        base = new();
        burst = new();

        base.randomize(); burst.randomize();
        base.print();    burst.print();

        // Polymorphism — base handle can point to child object

        poly_handle = burst; // child assigned to parent handle
        poly_handle.print(); // calls fifo_burst_txn's print()
                          // because print() is virtual
    end
endmodule

```

Output:

```

# KERNEL: Base txn: data=46 wr=1
# KERNEL: Burst txn: data=5f wr=0 len=4
# KERNEL: Burst txn: data=5f wr=0 len=4

```

The complete class based testbench for FIFO:

```

// tb_fifo_class.sv
`timescale 1ns/1ps
`include "fifo.sv"
// Import our transaction class

module tb_fifo_class;
    parameter DEPTH = 8;
    parameter WIDTH = 8;

    logic      clk, rst;
    logic      wr_en, rd_en;
    logic [WIDTH-1:0] din, dout;
    logic      full, empty;

    // Instantiate DUT — your Day 6 FIFO
    fifo #(DEPTH, WIDTH) dut(
        .clk(clk), .rst(rst),
        .wr_en(wr_en), .rd_en(rd_en),
        .din(din), .dout(dout),
        .full(full), .empty(empty)
    );

```

```

initial begin clk = 0; forever #5 clk = ~clk; end

initial begin
    $dumpfile("fifo_class.vcd");
    $dumpvars(0, tb_fifo_class);
end

// Drive one transaction to DUT
task drive(fifo_txn txn);
    @(negedge clk);
    wr_en = txn.wr_en & !full;
    rd_en = txn.rd_en & !empty;
    din = txn.data;
    @(posedge clk); #1;
    // Simple self-check for write
    if(txn.wr_en && !full)
        $display("Wrote: %h | full=%b empty=%b",
            txn.data, full, empty);
    if(txn.rd_en && !empty)
        $display("Read: %h | full=%b empty=%b",
            dout, full, empty);
    wr_en = 0; rd_en = 0;
endtask

initial begin
    fifo_txn txn;
    txn = new();

    rst = 1; wr_en = 0; rd_en = 0; din = 0;
    #20; rst = 0;

    $display("=== Random stimulus test ===");

    // Run 20 random transactions
    repeat(20) begin
        txn.randomize();
        txn.print("DRIVE");
        drive(txn);
    end

    // Drain remaining data
    repeat(DEPTH) begin
        @(negedge clk);
        rd_en = !empty;
        @(posedge clk); #1;
        if(!empty)
            $display("Drain: %h", dout);
        rd_en = 0;
    end

    $display("=== Done ===");
    #20; $finish;
end
endmodule

```

Output:

```
KERNEL: === Random stimulus test ===
# KERNEL: [DRIVE] data=ec wr=1 rd=0
# KERNEL: Wrote: ec | full=0 empty=0
# KERNEL: [DRIVE] data=06 wr=0 rd=1
# KERNEL: [DRIVE] data=75 wr=0 rd=1
# KERNEL: [DRIVE] data=96 wr=0 rd=1
# KERNEL: [DRIVE] data=aa wr=1 rd=0
# KERNEL: Wrote: aa | full=0 empty=0
# KERNEL: [DRIVE] data=7d wr=0 rd=1
# KERNEL: [DRIVE] data=a3 wr=0 rd=1
# KERNEL: [DRIVE] data=fe wr=1 rd=0
# KERNEL: Wrote: fe | full=0 empty=0
# KERNEL: [DRIVE] data=91 wr=0 rd=1
# KERNEL: [DRIVE] data=9c wr=0 rd=1
# KERNEL: [DRIVE] data=a6 wr=1 rd=0
# KERNEL: Wrote: a6 | full=0 empty=0
# KERNEL: [DRIVE] data=78 wr=0 rd=1
# KERNEL: [DRIVE] data=f3 wr=0 rd=1
# KERNEL: [DRIVE] data=4d wr=1 rd=0
# KERNEL: Wrote: 4d | full=0 empty=0
# KERNEL: [DRIVE] data=16 wr=0 rd=1
# KERNEL: [DRIVE] data=71 wr=1 rd=0
# KERNEL: Wrote: 71 | full=0 empty=0
# KERNEL: [DRIVE] data=13 wr=1 rd=0
# KERNEL: Wrote: 13 | full=0 empty=0
# KERNEL: [DRIVE] data=87 wr=0 rd=1
# KERNEL: Read: 71 | full=0 empty=0
# KERNEL: [DRIVE] data=23 wr=1 rd=0
# KERNEL: Wrote: 23 | full=0 empty=0
# KERNEL: [DRIVE] data=75 wr=0 rd=1
# KERNEL: Read: 13 | full=0 empty=0
# KERNEL: === Done ===
```